GUIDE

A Beginner's Guide to Software Development Using Coding Agents

Author

Bargava

11 October 2026

Contents

1 About this document . . . 2
1.1 Who this is for . . . 2
1.2 What you learn . . . 2
1.3 Writing conventions . . . 2
2 What software development using coding agents is . . . 2
3 Tools you need . . . 3
3.1 Required applications and accounts . . . 3
3.2 Claude Code plugins (skills) . . . 4
4 Set up your workstation . . . 4
4.1 Install the tools . . . 4
4.2 Sign in to the services . . . 4
4.3 Store secrets . . . 5
5 The skills you use . . . 5
5.1 Matt Pocock skills . . . 5
5.2 Impeccable skill . . . 7
6 The workflow you follow . . . 8
6.1 Step 1: Write a short brief . . . 8
6.2 Step 2: Configure the repository . . . 9
6.3 Step 3: Capture the product context . . . 9
6.4 Step 4: Sharpen the idea . . . 9
6.5 Step 5: Prototype the risky part . . . 10
6.6 Step 6: Write the spec and the tickets . . . 10
6.7 Step 7: Build one ticket at a time . . . 10
6.8 Step 8: Run it on real input . . . 11
6.9 Step 9: Deploy to Railway . . . 11
7 Documents that the process writes . . . 13
8 Use the standard order . . . 13
9 Rules for working with the agent . . . 14
9.1 Give context . . . 14
9.2 Keep control . . . 14
9.3 Protect secrets . . . 14
9.4 Verify the result . . . 14
10 Glossary . . . 15

1 About this document

1.1 Who this is for

This guide is for a junior developer. You choose the project. The agent writes most of the code. You make the decisions, and you check the result. Read the guide from start to end before you start.

1.2 What you learn

After you read this document, you can do these tasks:

- Install and configure the tools.
- Explain each phase of software development using coding agents.
- Use the Matt Pocock skills and the Impeccable skill in the correct order.
- Find the documents that the process writes, and know what each document controls.
- Ask the agent how to deploy, which services the project needs, and where an API key goes.

1.3 Writing conventions

- **Bold** text shows a name in a user interface or an important term.
- `Code font` shows a command, a file name, or a value that you type.
- A command that starts with `/` is a Claude Code slash command. You type it in the Claude Code prompt.
- A command that starts with `!` runs in the shell from the Claude Code prompt.

2 What software development using coding agents is

In ordinary software development, you write all the code. In software development using coding agents, an agent (Claude Code) writes most of the code. You still own the work. You do these tasks:

- You choose the project and the decisions.
- You give the agent clear context, in files.
- You approve the actions that are risky or that you cannot easily undo.
- You verify the result on real input.

The agent does the research, the writing, the coding, and the testing. You do not accept a result because the agent says it is done. You look at the evidence.

Skills give the agent a repeatable process. A skill is a packaged set of instructions for one type of task. One skill interviews you about a plan. Another skill turns that plan into tickets. You run the skills in order, on whatever project you choose.

The work has six phases:

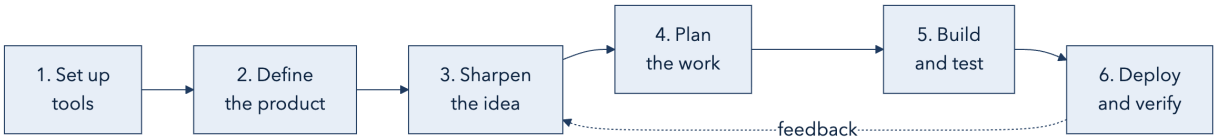


Figure 1. The six phases. Feedback returns to sharpening the idea.

Each phase writes documents in the repository. The next phase reads those documents. This is the most important rule: **the repository holds the context, not the chat.** A new session can continue the work because the decisions are in files.

3 Tools you need

3.1 Required applications and accounts

You install these once. The links open the product site.

Tool	Purpose	How you install it
Claude Code	The coding agent. It reads and writes files, runs commands, and runs skills.	The Claude Code installer. The executable is `claude`.
Orca	The terminal application where you run Claude Code.	The Orca installer.
Git	Version control. The agent commits only when you approve.	Included with the Windows developer setup.
GitHub account and repository	Hosts the code and the issue tracker.	Web sign-up. Create an empty repository for your project.
GitHub CLI (`gh`)	Lets the agent create and read GitHub issues.	`winget install -e --id GitHub.cli`
Railway account	Hosts the deployed application. It can also host a database.	Web sign-up.
Railway CLI (`railway`)	Lets you and the agent manage Railway from the terminal.	`npm i -g @railway/cli`
Railway MCP server	Gives Claude Code tools to manage Railway projects.	`claude mcp add --transport http railway https://mcp.railway.com`
Python	A language the agent can use for scripts and for a server.	`winget install -e --id Python.Python.3.12 --scope user`
uv	A fast Python package manager. Optional.	`irm https://astral.sh/uv/install.ps1 | iex`
Node.js LTS	Builds a web interface when the project has one. Includes `npm`.	`winget install -e --id OpenJS.NodeJS.LTS --scope user`
Docker (optional)	Builds a production image on your machine. Railway can build it for you.	Not required on your workstation.

3.2 Claude Code plugins (skills)

Plugin	Purpose
`mattpocock-skills`	The engineering process: interview, glossary, decisions, spec, tickets, implementation, review.
`impeccable`	Product context and user interface design.

4 Set up your workstation

You do these steps once on a new workstation.

4.1 Install the tools

1. Install Claude Code and Orca.
2. Open a terminal in Orca.
3. Install Python, Node.js, and the GitHub CLI for your user account only:

```PowerShell
winget install -e --id Python.Python.3.12 --scope user
winget install -e --id OpenJS.NodeJS.LTS --scope user
winget install -e --id GitHub.cli --scope user
```

> **Note.** Use `--scope user`. A machine-wide install shows a Windows administrator prompt (UAC). The agent cannot accept that prompt for you. If you see a UAC prompt, you click **Yes** or **No** yourself.

4. Install the Railway CLI:

```PowerShell
npm i -g @railway/cli
```

5. Close the terminal and open a new one. This loads the new `PATH`.
6. Make sure that each tool runs:

```PowerShell
python --version
node --version
gh --version
railway --version
```

4.2 Sign in to the services

1. Sign in to GitHub:

```Shell
! gh auth login
```

2. Sign in to Railway:

```Shell
! railway login
```

3. Add the Railway MCP server to Claude Code:

```PowerShell
claude mcp add --transport http railway https://mcp.railway.com
```

4. In Claude Code, type `/mcp`. Select **railway**, and then select **Authenticate**.
5. Restart Claude Code with `claude --continue`.

> **Important.** Claude Code loads MCP tools only when a session starts. After you add or authenticate an MCP server, you restart the session. A restart also stops background agents and local servers. Wait until background work is complete before you restart.

4.3 Store secrets

A secret is a key that lets your app call an outside service. You never put a key in the chat, and you never commit a key.

1. Ask the agent which service the project needs. For an app that parses a PDF, you can ask which service turns a PDF into fields, and what you must create on that service's website.
2. Create the key on that website. Put it in `.env.local` at the root of the repository:

```Text
PDF_API_KEY=<your key>
```

3. Make sure that `.gitignore` excludes the file:

```PowerShell
git check-ignore -v .env.local
```

4. When you deploy, add the same name as a Railway variable. The value stays in Railway. It does not go into the chat.

> **Warning.** Do not paste an API key into the chat. Do not commit an API key. Locally, the key lives in `.env.local`. In production, it lives in a Railway variable.

5 The skills you use

5.1 Matt Pocock skills

The Matt Pocock skills are the engineering process. They move an idea to shipped code on a fixed path. The skills keep the decisions in the repository, so the next session can continue.

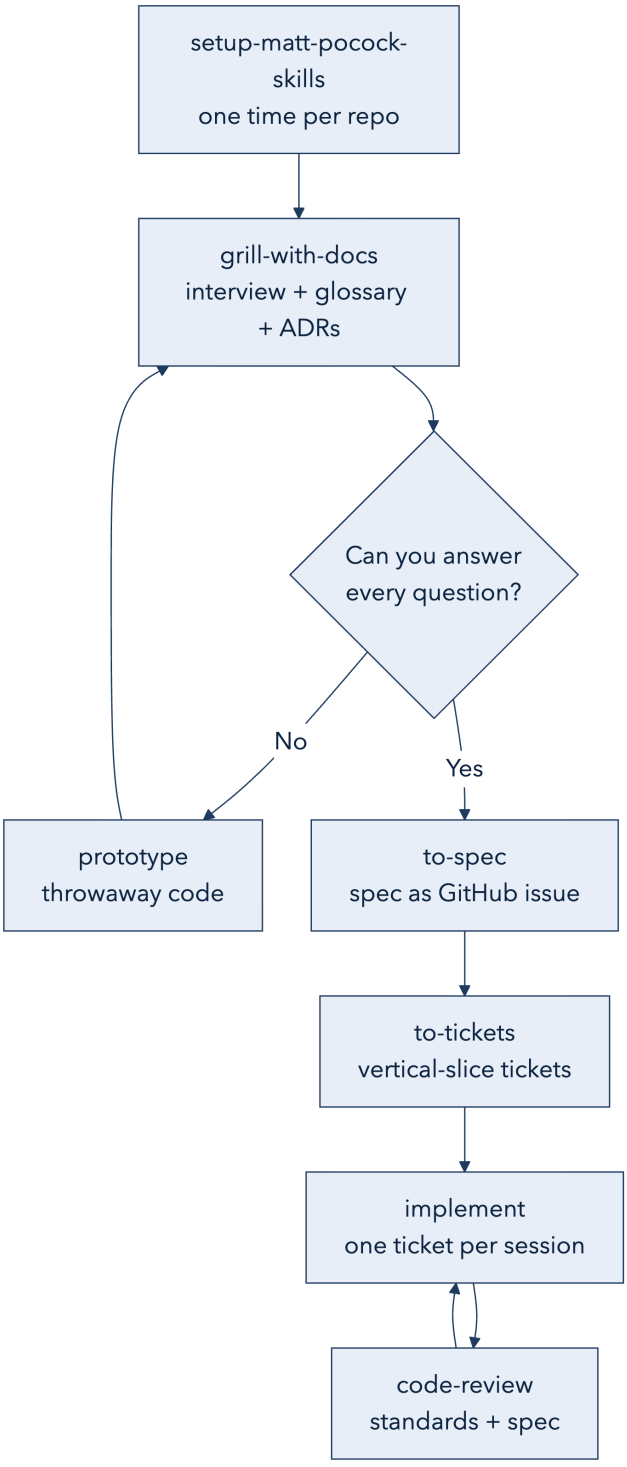


Figure 2. Matt Pocock skills, from setup through review.

Skill	What it does	What it makes	How you use it
`/setup-matt-pocock-skills`	Configures the issue tracker, triage labels, and the location of domain documents.	`CLAUDE.md`, `docs/agents/*.md`	You choose GitHub Issues and the default labels.
`/grill-with-docs`	Interviews you in rounds until the decisions are clear. It runs `grilling` and `domain-modeling` together.	`CONTEXT.md`, `docs/adr/*.md`	You answer each round. The decisions for your project land in files.
`/grilling`	The interview engine. Each round asks every question that you can answer now, with a recommended answer.	Decisions in the chat, then in files	You do not type this yourself. `grill-with-docs` runs it.
`/domain-modeling`	Keeps one name for each concept. Writes the glossary and the architecture decision records (ADRs).	`CONTEXT.md`, ADRs	You use the names from your project. You do not invent a second name for the same thing.
`/to-spec`	Turns the conversation into a specification.	A GitHub issue with user stories	You run it after the interview. You read the issue before you approve it.
`/to-tickets`	Splits the spec into tracer-bullet tickets. Each ticket cuts through every layer. Each ticket names what blocks it.	GitHub issues with blocking links	You approve the breakdown before any feature code is written.
`/implement`	Builds one ticket test-first, then runs a code review.	Code, tests, a commit	You run one ticket per session. See Use the standard order.
`/ask-matt`	Tells you which skill to use next.	Advice	You ask it when you are not sure of the next step.

> **Note.** Some skills (`/to-spec`, `/to-tickets`, `/implement`) run only when **you** type them. The agent cannot start them. Type the full name, for example `/mattpocock-skills:to-spec`, as a message by itself.

5.2 Impeccable skill

The Impeccable skill is the design process. It keeps the interface honest. It does not start with colors and fonts. It starts with the product.

Command	What it does	What it makes	How you use it
`/impeccable init`	Interviews you about users, purpose, and constraints. It does not ask about visual style.	`PRODUCT.md`	You describe who the app is for, what it does, and what it must not do.
`/impeccable` (new work)	Chooses a visual direction, builds the interface, and checks it against a craft floor.	`DESIGN.md`, `.impeccable/`	You run it after `PRODUCT.md` exists, when a ticket needs a screen.
`impeccable detect`	Scans the interface for known design problems.	A report	You run it before you call the interface finished.

Other commands, such as `critique`, `audit`, and `polish`, refine an interface that already exists. You use them after the first version works.

6 The workflow you follow

You follow these steps in order. The project can be anything you can describe in a few sentences. The examples below use an app that parses a PDF, so you can see how to ask. You replace that example with your own project.

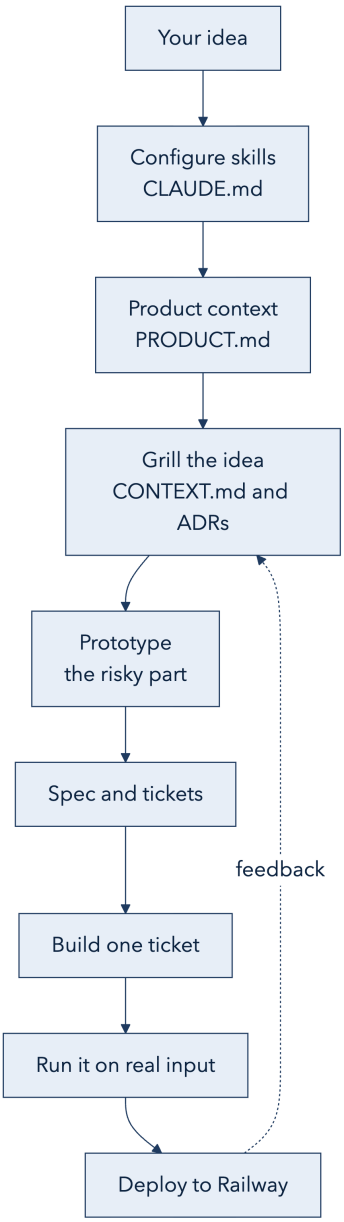


Figure 3. The order you follow on a first project.

6.1 Step 1: Write a short brief

Goal Say what you want to build, in one page.

You tell the agent the idea in plain language. You ask it to write `docs/productbrief.md`. The brief states the user, the outcome, what is out of scope, and the open questions.

For a PDF project, the whole brief can start from one sentence: ”A person uploads a PDF and sees the fields the parser found.”

6.2 Step 2: Configure the repository

Goal Tell the skills where issues and domain documents live.

1. Type `/mattpocock-skills:setup-matt-pocock-skills`.
2. Answer the questions. You choose:
   - Issue tracker: GitHub.
   - Triage labels: the defaults.
   - Configuration file: `CLAUDE.md`.

6.3 Step 3: Capture the product context

Goal Record the facts that do not change often.

1. Type `/impeccable:impeccable init`.
2. Answer the questions about the users, the purpose, and the name of your app.

The result is `PRODUCT.md`. Later design work reads this file.

6.4 Step 4: Sharpen the idea

Goal Make the decisions before you write feature code.

1. Type `/mattpocock-skills:grill-with-docs` and give a one-line scope. For the PDF example: ”A small app. A person uploads a PDF and sees the parsed fields.”
2. Answer each round. Each question has a recommended answer. You accept it in one word, or you change it.
3. Read each update to `CONTEXT.md` as the agent writes it.
4. Accept the ADRs that the agent offers.

You answer these kinds of questions. The exact questions depend on your project:

Question	What you decide	Where it goes
Who is it for?	The person who uses the app, and what they are trying to finish.	`PRODUCT.md`
What is in, and what is out?	The one outcome, and the things the first version does not do.	`PRODUCT.md`, `CONTEXT.md`
What is the risky part?	The piece you are not sure will work. For a PDF app, that piece is the parse.	A short prototype
What do you store?	What the app keeps, and what it throws away.	An ADR
Which outside service?	Which API does the hard work, and which key it needs.	An ADR and `.env.local`
How does it ship?	Railway, and which services (a web service, and a database only if you store data).	An ADR

> **Tip.** A decision can change before you commit it. You ask the agent to rewrite the ADR. After you commit an ADR, you add a new ADR that supersedes it. You do not silently edit the old one.

6.5 Step 5: Prototype the risky part

Goal Prove the hard part with a throwaway script, before you design the whole app.

You ask for a small experiment, not a product. For the PDF example, you type:

```text
Write a short script that sends one sample PDF to the parsing API and prints the fields. Do not build the we
```

You run the script on a real file. You read the fields. If they are wrong, you change the decision now. A prototype is supposed to be thrown away.

6.6 Step 6: Write the spec and the tickets

Goal Turn the decisions into work that someone else can track.

1. Type `/mattpocock-skills:to-spec`. Read the user stories. Confirm how you will test the app: through the real interface, and with one real input.
2. Type `/mattpocock-skills:to-tickets`. Read the breakdown. Approve it, or send it back.

Each ticket is a tracer bullet. It goes through every layer that the feature needs, and you can demonstrate it. Later tickets name the tickets that block them:

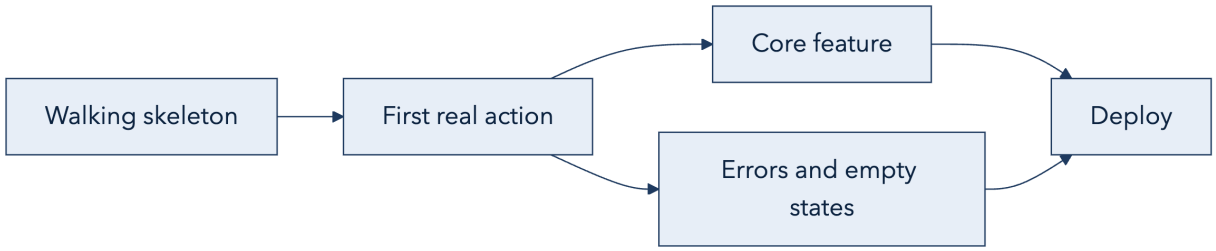


Figure 4. Later tickets wait until earlier tickets are done.

For an app that parses a PDF, the core feature is: upload a file and show the fields. You still use these slice names. You do not invent a different process for each project.

6.7 Step 7: Build one ticket at a time

Goal Ship a thin slice that you can run.

You name one ticket, then you send this as its own message:

```text
/mattpocock-skills:implement
```

The agent reads the ticket, `CONTEXT.md`, and the ADRs. It writes tests first. It shows you the test output. You approve the commit, or you send a correction.

You give a sub-agent a job only when the work splits into folders that do not touch the same files. You

tell it the folder it owns, the folders it must not touch, the tests to write, and what to report. On a first project, one ticket at a time is enough.

6.8 Step 8: Run it on real input

Goal Prove the slice on input that did not come from the tests.

1. Run the tests.
2. Start the app the way the ticket describes.
3. Use one real input. For a PDF app, that input is a PDF you did not use while writing the tests.
4. Compare the result with the file. If a value is not in the file, the app must not invent it.

> **Important.** You run the real system on real input before you deploy. Tests with fakes cannot find the mistakes of a model or of an outside API.

6.9 Step 9: Deploy to Railway

Goal Put the app on a public URL, with secrets kept out of the chat.

You do not memorize Railway's screens. You ask the agent, and you check its plan before it creates anything. For the PDF example, you type:

Text

```
Deploy this project to Railway. It parses a PDF with an external API. Which Railway services does it need?
```

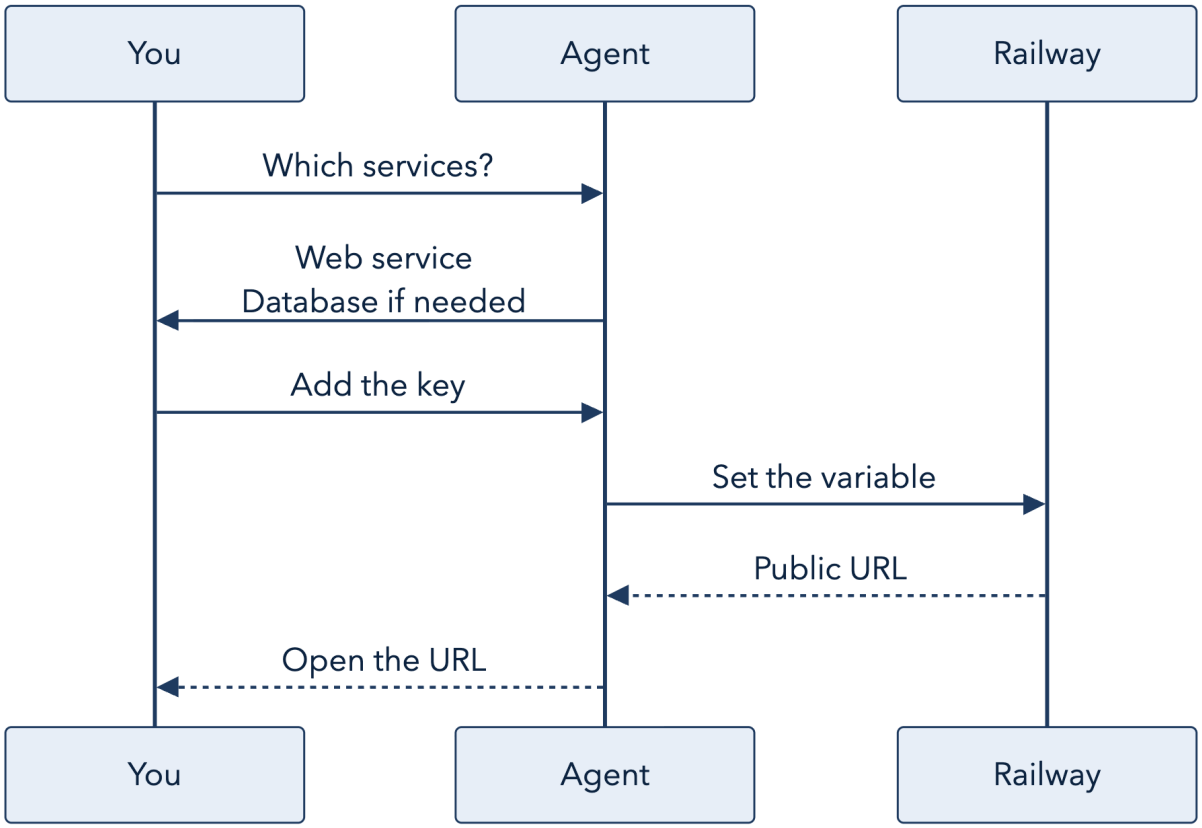


Figure 5. How you ask the agent to deploy. You check the public URL yourself.

1. Restart Claude Code so the Railway MCP tools load.
2. Read the list of services. A web service is normal. A database is only for data you decided to store.
3. Create the API key on the provider's website. Add it in Railway under the name from `.env.local`. You do this yourself, or you approve a tool call that sets the variable without showing you the value in the transcript.
4. Approve the deploy.
5. Open the public URL and try one real file. Check the values by hand.

7 Documents that the process writes

File	Owner	What it controls
`CLAUDE.md`	Matt Pocock setup	Where issues and domain documents are. The agent reads it in every session.
`docs/agents/*.md`	Matt Pocock setup	Issue tracker commands, triage labels, and domain document rules.
`PRODUCT.md`	Impeccable init	Users, purpose, principles, and constraints.
`DESIGN.md`	Impeccable	Visual system: colors, type, layout, and components.
`CONTEXT.md`	Domain modeling	The glossary. One name for each concept.
`docs/adr/*.md`	Domain modeling	Decisions that are hard to reverse, and why you made them.
`docs/productbrief.md`	You and the agent	The first description of the idea. Later files replace it when decisions change.
The specification issue	`to-spec`	The specification and the user stories.
The tickets	`to-tickets`	The work items, with acceptance criteria and blockers.

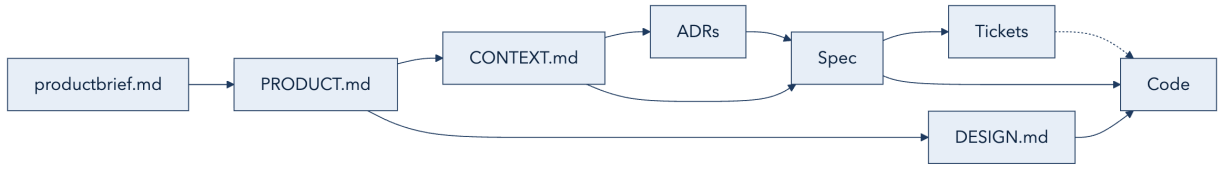


Figure 6. Documents the process writes, and which ones the code reads.

8 Use the standard order

On a first project, you follow the skill order. You do not build the whole app and write the tickets afterwards.

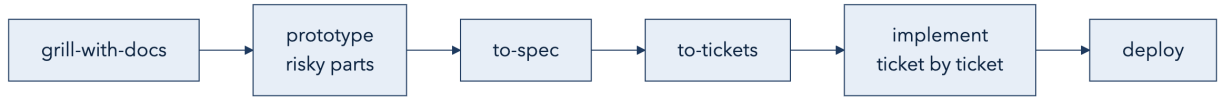


Figure 7. The standard order for a first project.

- You prototype the risky part. A late surprise is more expensive than a thrown-away script.
- You write the spec, then the tickets, then you implement. The tickets drive the build.
- You let `implement` run a code review for each ticket. You read the review.

9 Rules for working with the agent

9.1 Give context

- You put decisions in files, not only in the chat.
- You use the names from `CONTEXT.md`. You do not invent a second name for the same thing.
- You give a sub-agent one task, one folder, and the documents to read.

9.2 Keep control

- You approve every action that you cannot easily undo: commits, pushes, deploys, and deletions.
- You accept a Windows UAC prompt yourself. The agent cannot see it.
- You type the user-only skills yourself. The agent cannot start them.

9.3 Protect secrets

- You keep keys in `.env.local` or in Railway variables.
- You make sure that `.gitignore` excludes every `.env*` file before the first commit.
- You do not paste a key into the chat. You ask the agent where the key goes, not to repeat the key.

9.4 Verify the result

- You read the tests. A test checks behavior, not private details inside the code.
- You run the real system on real input.
- When the agent reports a result, you check the evidence: test output, the screen, and the logs.

10 Glossary

Term	Definition
ADR	Architecture decision record. A short file that records one decision and the reason for it.
Agent	A program that reads files, runs commands, and writes code for you. In this guide, that program is Claude Code.
MCP	Model Context Protocol. A standard way to give the agent tools for an outside service, such as Railway.
Skill	A packaged set of instructions for one type of task.
Sub-agent	A separate agent that the main agent starts for one task. It has its own context.
Tracer bullet	A thin ticket that goes through every layer the feature needs, so that you can demonstrate it.
UAC	User Account Control. The Windows prompt that asks for administrator permission.
Variable	A named setting, such as an API key. Locally it lives in `.env.local`. In production it lives in Railway. It does not live in the chat.